Zonoth

SESSION 03 · 2H

Running containers well

Configuration, storage and networking — everything between "the image builds" and "the app actually runs".

Configuration & signals | Storage | Networking | Limits & health | Lab 03

SESSION 03

What we'll cover today

1 · Configuration & process

`ENV`, `-e`, `--env-file`, `ARG` vs `ENV`, `CMD` vs `ENTRYPOINT`, exec vs shell form, PID 1 and SIGTERM.

2 · Storage

Why the writable layer disappears; volumes vs bind mounts vs tmpfs; the `--mount` syntax; volume lifecycle and backups.

3 · Networking, limits, health

User-defined bridges and DNS by container name, port publishing, `--memory` / `--cpus`, restart policies, `HEALTHCHECK`.

4 · Lab 03

Wire the whole Pinboard stack together by hand on `pinboard-net` with the `pinboard-data` volume — no Compose yet.

01

Configuration & the container process

One image, many environments — and one process that has to start and stop cleanly.

12-FACTOR

Configuration comes from the environment

One `pinboard-api:1.0` image everywhere; only the environment changes.

- `ENV` in the Dockerfile = the **default**
- `-e KEY=value` overrides it per container
- `--env-file` for a whole set — no quoting, no shell expansion
- Never bake secrets into the image

BASH

```
  DB=postgres://pinboard:pinboard-secret@db:5432/pinboard
docker run -d --name api \
  -e DATABASE_URL="$DB" -e APP_THEME=amber \
  pinboard-api:1.0

# or the same set from a file
docker run -d --name api --env-file ./api.env pinboard-api:1.0

docker inspect api --format '{{json .Config.Env}}'
```

DON'T CONFUSE THEM

ARG vs ENV

ARG	ENV
Build time only	Build time **and** run time
Gone from the final image	Baked into the image config
`--build-arg`	`-e` / `environment:`

⚠ An `ARG` value is visible in `docker history`. Never pass a password that way — use `--mount=type=secret`.

DOCKERFILE

```
ARG APP_VERSION=dev

ENV PORT=8080 \
    APP_VERSION=${APP_VERSION}

ENTRYPOINT ["/pinboard-api"]
```

BASH

```
docker build -t pinboard-api:1.1 \
--build-arg APP_VERSION=1.1 ./pinboard-api
```

CMD vs ENTRYPOINT

- `ENTRYPOINT` — the **executable**. Makes the image behave like a command.
- `CMD` — the **default arguments**, easy to replace.
- Arguments after the image name replace `CMD`, never `ENTRYPOINT`.
- `--entrypoint` overrides the executable itself.

DOCKERFILE

```
ENTRYPOINT ["/pinboard-api"]
CMD ["--log-format=text"]
```

BASH

```
# runs: /pinboard-api --log-format=text
docker run pinboard-api:1.0

# runs: /pinboard-api --log-format=json
docker run pinboard-api:1.0 --log-format=json

# debug a distroless image? you can't shell in:
docker run --entrypoint /bin/sh alpine -c 'echo hi'
```

THE FORM MATTERS

Exec form vs shell form

DOCKERFILE

```
# shell form → /bin/sh -c "/pinboard-api"
CMD /pinboard-api

# exec form → the binary itself
CMD ["/pinboard-api"]
```

Always use the JSON array (exec) form for `CMD` and `ENTRYPOINT`.

Shell form

`sh` is PID 1. Your process is a child. `sh` does not forward signals, and variables like `$PORT` **are** expanded.

Exec form

Your process is PID 1 and receives signals directly. No shell, no expansion — and one less process in the image.

SHUTDOWN

PID 1, SIGTERM and graceful stop

01
docker stop
You (or the orchestrator) ask the container to stop

→

02
SIGTERM to PID 1
Only PID 1 gets it — nobody else

→

03
Grace period
Default 10 s (`--stop-timeout`) to drain

→

04
SIGKILL
Unconditional. Exit code 137

- Shell form ⇒ `sh` ignores SIGTERM ⇒ every stop takes the full 10 s and ends in a kill
- PID 1 has no default signal handlers and never reaps orphans — `--init` inserts a tiny init for that
- Pinboard API handles SIGTERM and drains for `SHUTDOWN_GRACE` (default `10s`)

LIVE DEMO

Signals: shell form vs exec form

Stop the same API twice — once wrapped in a shell, once as PID 1 — and watch the clock and the exit code disagree.

What you'll see · `docker top`, a 10 s stop vs an instant one, exit 137 vs 0 **Files** · src/pinboard-api **Duration** · 6 min

02

Storage

Containers are disposable. Your data must not be — the writable layer dies with the container.

The writable layer is ephemeral

01 — Image layers
Read-only, shared by every container from that image

+

02 — Writable layer
Copy-on-write, created per container

→

03 — docker stop / start
Layer survives — data is still there

→

04 — docker rm
Layer deleted — data gone forever

- Writes go through an overlay driver: slower than a real filesystem, and unusable for a database's fsync patterns
- Anything you want to keep — or share between containers — must be **mounted** from outside

⚠ "The container restarted and my data is gone" is almost always `docker rm` plus no volume.

THREE WAYS TO MOUNT

Volumes vs bind mounts vs tmpfs

VOLUME	BIND MOUNT	TMPFS
Docker owns it (`/var/lib/docker/volumes`)	Any path on the host, you own it	Host RAM only, never on disk
Named, portable, backed by plugins/NFS	Path must exist as you expect on every host	Dies with the container
Best for data: databases, uploads	Best for **dev**: source and config live-editing	Best for **secrets** and scratch files
`pinboard-data` **for Postgres**	`./pinboard-web/html` while styling	`/tmp`, session caches

Default to **volumes**. Reach for a bind mount when you specifically want the host's files.

SYNTAX

Learn `--mount`, not `-v`

BASH

```
 docker volume create pinboard-data

docker run -d --name db --network pinboard-net \
  --mount type=volume,src=pinboard-data,dst=/var/lib/postgresql/data \
  -e POSTGRES_USER=pinboard -e POSTGRES_PASSWORD=pinboard-secret \
  -e POSTGRES_DB=pinboard postgres:17-alpine

docker run --rm -p 8081:8080 --name web-dev \
  --mount type=bind,src="$PWD/pinboard-web/html",dst=/usr/share/nginx/html,ro \
  pinboard-web:1.0

docker run --rm --mount type=tmpfs,dst=/tmp,tmpfs-size=64m alpine df -h /tmp
```

✓ `-v host:container` still works, but it **creates** missing host paths silently and hides the mount type. `--mount` is explicit and fails loudly.

HOUSEKEEPING

Volume lifecycle

- Volumes outlive containers — that is the whole point
- `docker rm` keeps them; `docker rm -v` only removes **anonymous** ones
- Anonymous volumes (no `src=`) pile up invisibly — always name them
- `docker volume prune` deletes everything unused: read the prompt

BASH

```
 docker volume ls
docker volume inspect pinboard-data
docker volume rm pinboard-data      # fails while in use

docker ps -a --filter volume=pinboard-data
docker system df -v                 # what is eating the disk
docker volume prune                 # unused volumes only
```

Backup with a throwaway container

BASH

```
  # tar the volume out to the current host directory
docker run --rm \
  --mount type=volume,src=pinboard-data,dst=/data \
  --mount type=bind,src="$PWD",dst=/backup \
  alpine tar czf /backup/pinboard-data.tgz -C /data .

# restore into a fresh volume
docker volume create pinboard-data-restored
docker run --rm \
  --mount type=volume,src=pinboard-data-restored,dst=/data \
  --mount type=bind,src="$PWD",dst=/backup \
  alpine tar xzf /backup/pinboard-data.tgz -C /data
```

For a database, prefer the application-aware dump: `docker exec db pg_dump -U pinboard pinboard > dump.sql`.

LIVE DEMO

Postgres with and without a volume

Lose the notes, then keep them — and bind-mount the Pinboard web folder to edit the page with the container running.

What you'll see · `rm` without a volume vs with `pinboard-data`; live HTML edit

Files · src/db, src/pinboard-web/html

Duration · 8 min

03

Networking, limits and health

How containers find each other, how you cap what they can consume, and how Docker knows they are alive.

DRIVERS

Network drivers you actually use

bridge (default)

The `docker0` network every container joins if you say nothing. Works — but **no DNS** between containers.

User-defined bridge

You create it. Embedded DNS resolves **container names**, and only attached containers can reach each other.

host

No network namespace at all: the container uses the host's stack directly. Fast, zero isolation, Linux only.

none

Loopback only. For batch jobs that must not talk to anything.

SERVICE DISCOVERY

Containers find each other by name

BASH

```
 docker network create pinboard-net

docker run -d --name db --network pinboard-net \
  --mount type=volume,src=pinboard-data,dst=/var/lib/postgresql/data \
  -e POSTGRES_USER=pinboard -e POSTGRES_PASSWORD=pinboard-secret \
  -e POSTGRES_DB=pinboard postgres:17-alpine

docker run -d --name api --network pinboard-net \
  -e DATABASE_URL=postgres://pinboard:pinboard-secret@db:5432/pinboard \
  -e APP_THEME=emerald pinboard-api:1.0

docker run -d --name web --network pinboard-net -p 8080:8080 \
  -e API_URL=http://api:8080 pinboard-web:1.0
```



`db`, `api` and `web` are DNS names on `pinboard-net`. No IP addresses anywhere.

THE DOOR TO THE OUTSIDE

Publishing ports

- `-p HOST:CONTAINER` — only the host side is negotiable
- Containers on the same network talk on the **container** port with no `-p` at all
- `EXPOSE` in a Dockerfile is documentation, not a firewall rule
- Bind to `127.0.0.1` to keep a port off the LAN

BASH

```
 -p 8080:8080              # all interfaces
-p 127.0.0.1:5432:5432     # localhost only
-p 8080:8080/udp           # protocol
-P                         # every EXPOSE → random port
```

BASH

```
 docker port web
docker ps --format '{{.Names}}\t{{.Ports}}'
```

Resource limits and the OOM killer

- By default a container may use **all** the host's CPU and RAM
- `--memory` is a hard cap enforced by cgroups: exceed it and the kernel kills the process
- An OOM-killed container exits with **137** and `OOMKilled=true`
- `--cpus` is a quota, not a priority — the process is throttled, not killed

BASH

```
docker run -d --name api \
 --memory=128m --memory-swap=128m \
 --cpus=0.5 pinboard-api:1.0

docker stats --no-stream
docker inspect api \
 --format '{{.State.OOMKilled}} {{.State.ExitCode}}'
```

KEEP IT RUNNING

Restart policies

POLICY	BEHAVIOUR	USE IT FOR
`no`	Default. Never restarted.	One-shot jobs, experiments
`on-failure[:3]`	Restart on a non-zero exit, up to N times	Batch work that may flake
`always`	Always restart, including after a daemon restart	Long-running services
`unless-stopped`	Like `always`, but respects a manual `docker stop`	What Pinboard uses

⚠ A restart policy reacts to the process **exiting**. An unhealthy-but-alive container is never restarted by Docker — that is an orchestrator's job.

IS IT ACTUALLY WORKING?

HEALTHCHECK

- States: `starting` → `healthy` / `unhealthy`
- `docker ps` shows it in the STATUS column
- Define it in the Dockerfile, or override with `--health-cmd` at run time
- Compose can **wait** on it — session 04

DOCKERFILE

```
HEALTHCHECK --interval=10s --timeout=2s --retries=3 \
 CMD wget -qO- http://127.0.0.1:8080/healthz || exit 1
```

BASH

```
docker ps
# STATUS  Up 40 seconds (healthy)

docker inspect web \
 --format '{{.State.Health.Status}}'
```

LIVE DEMO

Limits, restarts and the health column

Squeeze the API into 16 MB, watch it get killed, then bring the whole stack up on `pinboard-net` and read `docker ps`.

What you'll see · exit 137, `docker stats`, restart backoff, `(healthy)`

Files · src/compose.yaml services, by hand

Duration · 7 min

LAB 03 · 30–40 MIN

Wire Pinboard together by hand

Create `pinboard-net` and `pinboard-data`, then run all three tiers so they find each other by name — and prove the notes survive the death of the database container.

You'll practise: `network create` · `volume create` · `--mount` · `DATABASE_URL` by name · `API_URL` · `--memory` · health & restart policies

SESSION 03

Key takeaways

✓ Configure at run time

One image everywhere; `-e` / `--env-file` change behaviour. `ARG` is build time only.

✓ Exec form or you lose signals

Your process must be PID 1 to receive SIGTERM and shut down gracefully.

✓ State lives in volumes

The writable layer dies with the container. Named volumes for data, bind mounts for dev.

✓ Next up

Session 04 — every flag from Lab 03, declared once in `compose.yaml`.

Zonoth

Questions?

Up next — Session 04: Docker Compose.
Same stack, one file.